

Part 1: Predictions A to F

A. Event registration

- **When Hello appears:** Hello appears **immediately** when the line of code runs (at script execution time), because greet() calls the function instantly instead of passing it as a callback.
- **Will a later click call greet?** No, because addEventListener receives undefined (the return value of greet()), so clicking does nothing.

```
button.addEventListener("click", greet);
```

B. Independent closures

- **Predicted values:** 1, 2, 1, 3
- **Explanation:** makeCounter() creates a new function scope with its own distinct n variable each time it is called. a and b reference two independent closure instances. The third call executes b(), which increments b's separate counter from 0 to 1, whereas the second call incremented a's counter from 1 to 2.

C. Event loop order

- **Output order:** A, B, C, D
- **Identification:**
 - **Synchronous logs:** A and B
 - **Promise microtask:** C
 - **Timer task (macrotask):** D
- **Why zero delay does not mean immediate execution:** setTimeout places its callback into the **macrotask queue**. The JavaScript runtime must first finish executing all synchronous code (call stack) and clear all pending microtasks (Promise resolution handlers) before processing macrotasks.

D. Promise chaining

- **Logged value:** undefined

```
Promise.resolve(3)
```

```
.then(n => { return n * 2; }) // or implicitly: .then(n => n * 2)  
.then(value => console.log(value));
```

- **Role of return:** A `.then()` callback must return a value or promise to pass data down the chain to the next `.then()` block. Without return, the block implicitly returns undefined.

E. Rejection and cleanup

- **Output order:** outside, Offline, cleanup
- **Does success appear?** No, because `await Promise.reject(...)` throws an error, causing execution to jump straight into the catch block.
- **Why outside executes first:** `test()` is an async function. When it encounters `await`, it pauses execution of its body and returns a pending Promise back to the main thread, allowing the remaining synchronous code outside the function (`console.log("outside")`) to run immediately.

F. Class inheritance

- **What happens:** Uncaught ReferenceError: Must call super constructor in derived class before accessing 'this'.

```
class TimedResource extends Resource {  
  constructor(title, minutes) {  
    super(title);  
    this.minutes = minutes;  
  }  
}
```

- **Why order matters:** In JavaScript class inheritance, the derived class constructor must invoke `super()` before initializing its own properties or referencing `this`, because `super()` is responsible for instantiating the base object instance.

Part 3: Individual Exit Responses

1. **Which variable is retained by the counter closure, and why does its value survive between accepted requests?** The inner function retains access to the `count` variable declared inside `makeCounter()`. Its value survives across multiple calls because the returned function maintains an active reference to its lexical environment scope, preventing `count` from being garbage collected when `makeCounter()` finishes execution.

2. **Why do B and C appear before D in the execution trace?** Log B runs synchronously as part of the immediate thread. Log C is scheduled via `Promise.resolve().then()`, which enters the **microtask queue** and executes immediately after synchronous execution ends. Log D is scheduled via `setTimeout`, which goes to the **macrotask queue** and must wait for all call stack tasks and microtasks to complete first.
3. **What exactly pauses at `await`, and what can still happen while the request is pending?** Only the execution of the async form handler function pauses at the `await` keyword. The browser's main UI thread remains unblocked, allowing user interactions like scrolling, rendering, and handling independent events (such as the "Test page response" button click) to execute normally.
4. **How do `catch` and `finally` differ when a request fails?** The `catch` block executes conditionally **only** when a Promise rejection or runtime error is encountered, receiving the thrown error object. The `finally` block executes **unconditionally** after either `try` completes or `catch` handles an error, guaranteeing that cleanup steps (like resetting flags and re-enabling controls) are performed.
5. **What do `extends`, `super(title)`, and `super.describe()` each contribute to `TimedResource`?** `extends` establishes class inheritance linking `TimedResource` to `Resource`. `super(title)` executes the constructor of `Resource` to initialize inherited properties. `super.describe()` calls the parent method inside the overridden `describe()` method so that base title formatting can be reused and extended with minute metrics.